



Form3 Design

Integrated Web-Based Booking System and AI-Powered Guest Assistance with Proactive Cyber Defense Architecture

GROUP MEMBER:

No.	Student Name	Student ID
1.	Yonathan Marcello s. Tambani	001202300053
2.	Immanuel Ardiyanto	001202300057
3.	Muh. Fakhri Hisyam Akbar	001202300093

Advisor : Abdul Ghofir, S.Kom., M.Kom.

Submitted for
Capstone Design Project
to Faculty of Computer Science
President University

TABLE OF CONTENT

TABLE OF CONTENT.....	1
STATEMENT OF ORIGINALITY.....	2
SCREENSHOT OF TURNITIN.....	3
CHAPTER I SYSTEM DESIGN.....	4
1.1 Tier 1: Presentation Layer (The System Interface).....	4
1.2 Tier 2: Application Layer (The System Brain).....	4
1.3 Tier 3: Data Tier (Data Storage).....	5
1.4 Security Infrastructure (Wazuh / SIEM Integration).....	5
CHAPTER II HIERARCHICAL/ITERATIVE DESIGN.....	6
2.1 Data Flow Diagram.....	6
2.2 Block Diagram Level 0.....	7
2.3 Block Diagram Lvl 1.....	8
2.4 Block Diagram Level 2.....	8
2.5 Use Cases Diagram.....	9
CHAPTER III STANDARDS USED.....	12
3.1 List of the Standard.....	12
CHAPTER IV IMPLEMENTATION AND TESTING SCENARIO.....	14
4.1 Implementation Description.....	14
4.2 Testing Scenarios.....	14
REFERENCES.....	18

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone Design Project stated below:

Name : 1. Yonathan Marcello s. Tambani – 001202300053
2. Immanuel Ardiyanto – 001202300057
3. Muh. Fakhri Hisyam Akbar– 001202300093

Faculty : Computer Science

I hereby declare that my Capstone Design Project entitled “**Integrated Web-Based Booking System and AI-Powered Guest Assistance with Proactive Cyber Defense Architecture**” is to the best of my knowledge and belief, an original piece of work based on sound academic principles. If there is any plagiarism detected in this final project, I am willing to be personally responsible for the consequences of these acts of plagiarism and will accept the sanctions against these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another university to obtain a degree.

Cikarang, January 2024

Signer 1	Signer 2	Signer 3
		
Yonathan Marcello s. Tambani – 001202300053	Immanuel Ardiyanto – 001202300057	Muh. Fakhri Hisyam Akbar – 001202300093

SCREENSHOT OF TURNITIN



*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



CHAPTER I

SYSTEM DESIGN

1.1 Tier 1: Presentation Layer (The System Interface)

This is the visible and usable part of the users. Performance is lightning fast and highly responsive thanks to React 18 & Vite. To minimize confusion, we've divided it into two different "entry points":

1. Guest Portal: Web application that is visible to guests for checking and booking rooms.
2. Chatbot update: We added a chat widget on this page that enables guests to quickly check availability or hotel services. And this bot has "memory," letting it remember conversations even if visitors went to other pages.
3. Staff Dashboard (Admin, Receptionist & HRD) A personal command center for hotel team This will only get you access in this area for daily operations like; Check-ins, rate changes and the night audit.

1.2 Tier 2: Application Layer (The System Brain)

This is the "engine room" where all gets decided. Python and the FastAPI framework, This sermon is a gift.

1. Instant Response: Our backend only returns raw data (JSON) and not entire web pages (which our backend does) JWT (JSON Web Token) authenticators must be attached to every data requesting in order for security. Data can only be accessed with a token.
2. Modular Code: We separate the logic to files like auth etc.py hotel_reservation py for booking logic.
3. Technical Update Chatbot: New NLP Service In this way, the chatbot can directly "talk" with our reservation data and provide guests answers on the fly without any staff intervening.

1.3 Tier 3: Data Tier (Data Storage)

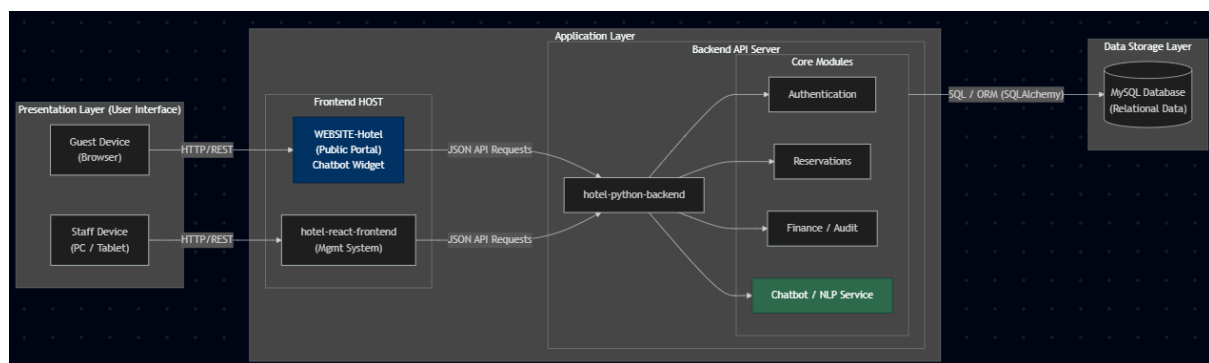
All important data which are cousin with a MySQL database is occur in this area securely.

1. Prevention of SQL Injection: None of the interaction with database. We employ SQLAlchemy ORM, which is a type of “bridge” between the two. In addition, it greatly helps to prevent malicious attacks such as SQL Injection.
2. Updated: Saving the conversation history to the database. This will allow us to measure customer satisfaction and continuously improve the quality of bot responses.

1.4 Security Infrastructure (Wazuh / SIEM Integration)

Users can't see it directly but we are performing a lot of monitoring with Wazuh. The Wazuh Agent runs quietly to the background and it can:

1. Constantly check for suspicious changes in the files
2. Identify Brute Force Attack on our API
3. Warn if there is a hole in the software we trust.



System Design Diagram: Presentation, Application, and Data Storage Layers

CHAPTER II

HIERARCHICAL/ITERATIVE DESIGN

2.1 Data Flow Diagram

A very high-level Data Flow Diagram has been called as Context Draw, is a general view of Integrated Hotel ManagementSystem. You can then formulate the system boundary of an application exactly as one, central authorization entity. This diagram shows how non-coupled entities exchange with the system at a conceptual level, emphasizing an aloha of "what" kind of Information is exchanged rather than internal processing details on "how" that process happens.

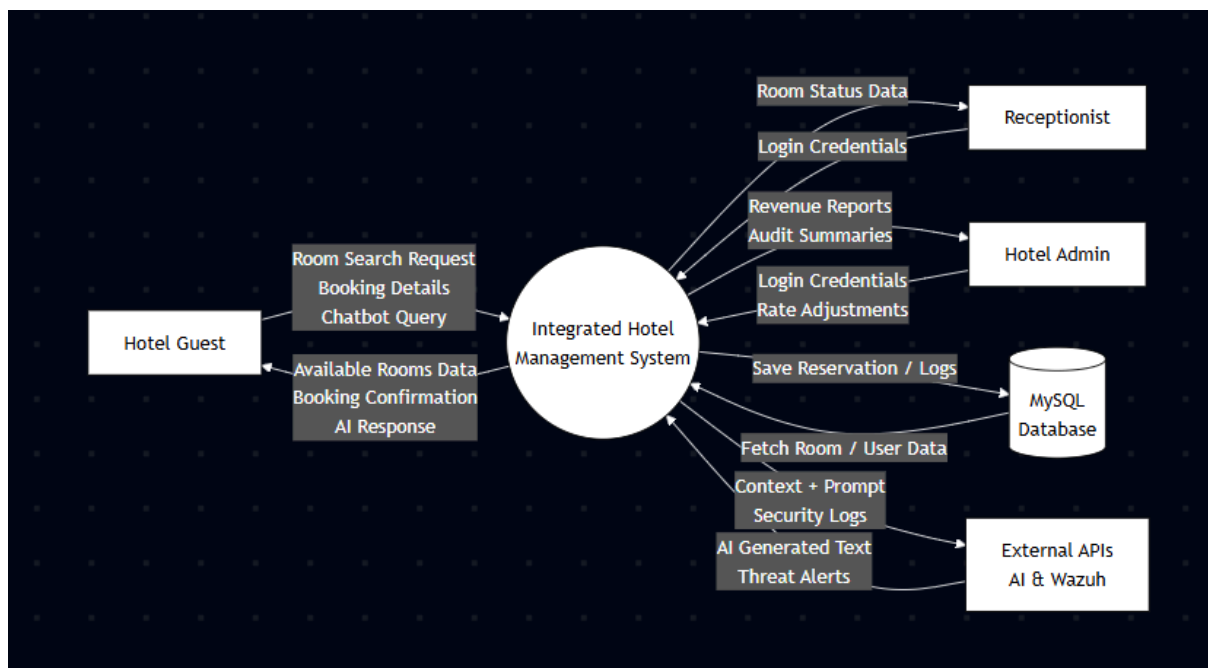


Figure 2.1

This architecture classifies external entities into human (Hotel Guest, Receptionist and Administrator) and external digital services (MySQL Database, AI Engine and Wazuh SIEM). The following diagram shows the primary input and output feeds detailed for entity. It shows staff and guests entering their user credentials and booking details, the system generating room states and confirmation information, etc. It further demonstrates the high-test power of the application ctx prompts automagically pushed contexts to an external

AI api as well as how application logs are routed every log frequency into a bifrequentious proctive threat monitor.

2.2 Block Diagram Level 0

For example Integrated Hotel Management represent this Integration System Relations Diagram visualizing a high-level integration architecture. It abstracts the entirety of big tech pillars React front end, Fast API back end + MySQL DB, external AI engine, Wazuh SIEM into macro parts. At a high level, this viewpoint is essential for understanding the network topology of distributed computing and how computation ends up being spread across an entire stack of technology which appears to be distributed over many nodes.

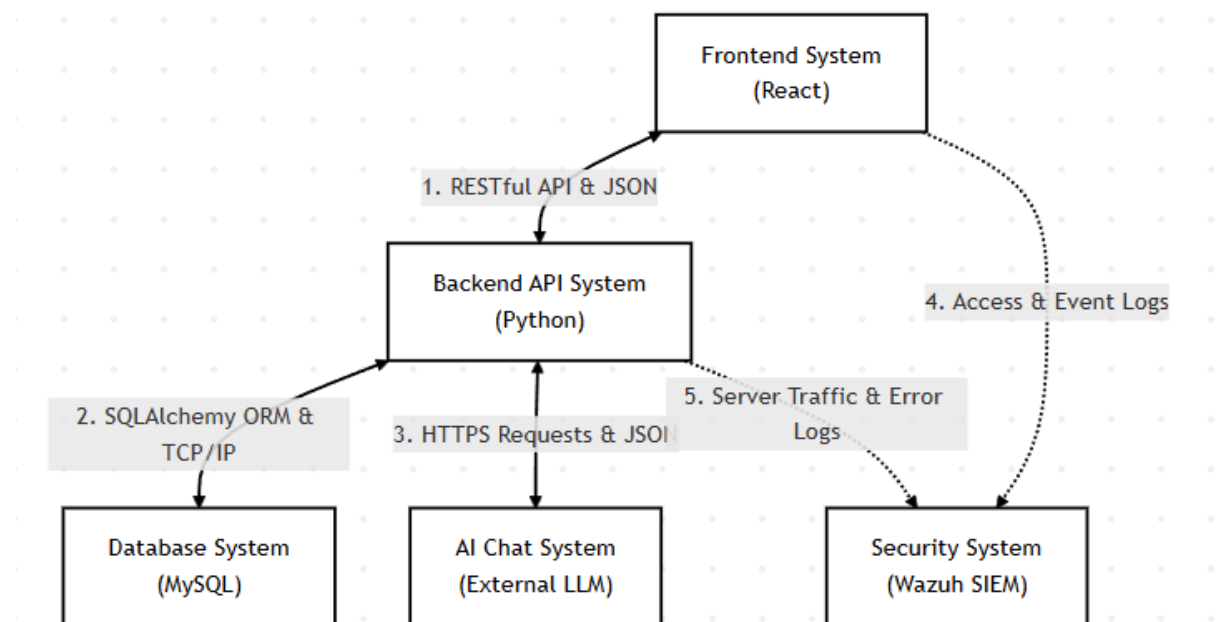


Figure 2.2

Beyond the structural layout, this diagram also dictates the communication protocols that bind these heterogeneous systems together. It proves that the frontend and backend exchange information over RESTful APIs (e.g., JSON payloads) and the backend talks to a database via secure TCP/IP connections on top of SQLAlchemy ORM. It also shows how external HTTPS requests that the AI chatbot should make as well as uni-directional, encrypted log forwarding to the proactive security monitor are configured such that this architecture is smoothly integrated yet still very loosely coupled.

2.3 Block Diagram Lvl 1

The Subsystem to System Diagram essentially articulates the functional scope of the core app by demonstrating how internal modules sit under a common umbrella platform. It acts as the Unified Boundary that will describe the Integrated Hotel Management System and seeing the front end single page application and back end API gateway as functional layers. Their canonical perspective on these layers explicitly defines what pieces are in the application control space (not externally referenced outside of the app's direct concern).

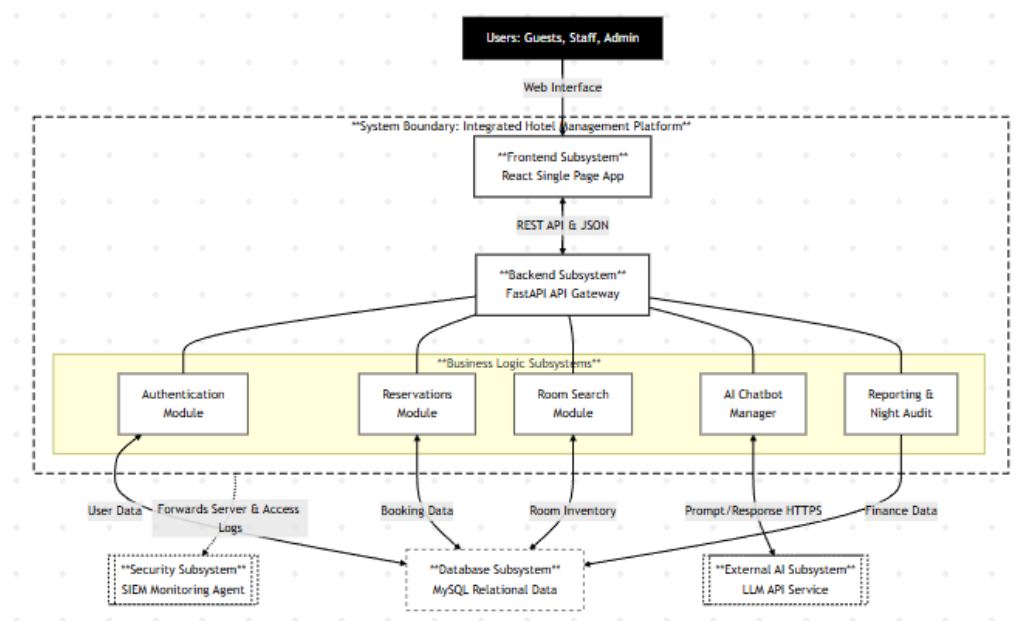


Figure 2.3 Diagram Level 3

What it also fails to convey is that internal logic (like authentication, reservation or chat bot modules) isolation increasingly depends on the parent system's infrastructure for outward availability. It separates the business logic from these external dependencies such as user web browser, relational database and Wazuh security agent. This is a good separation of concerns: the subsystems are doing some business rules and the whole system takes care of secure routing as well as any other integrations that need to occur to process them.

2.4 Block Diagram Level 2

Subsystem to Subsystem Diagram (also known as Component Interaction Diagram) describes in more detail the internal data flow between functional modules. Instead of treating the backend as a single giant monolithic block, the drawing separates individual business

logic subsystems e.g. Auth & Identity, Reservation Processing, AI Chat Manager, etc. Now, the fact that there is this level of detail to documentation of software engineering processes is significant as it shows that the application itself written in a very modular & maintainable micro-architecture.

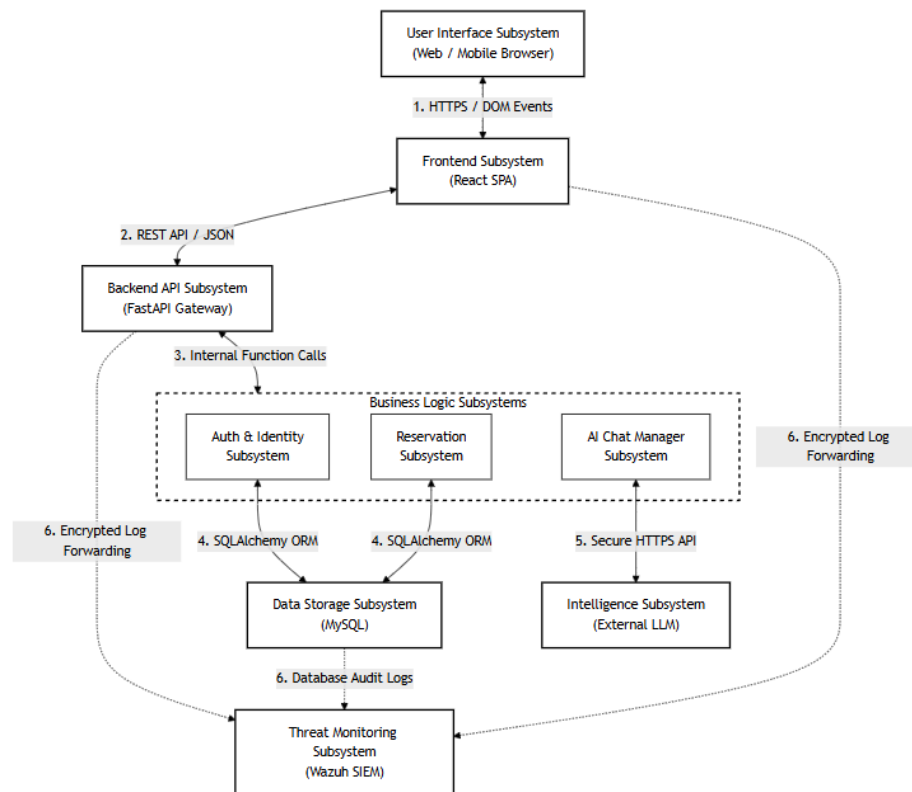


Figure 2.4 Diagram Level 2

This diagram illustrates these operational pathways between the localized components and highlights all dependencies that must be satisfied to process a user request. This illustrates the user interface subsystem calling the API gateway, which then delegates internal function calls to the specialised logic subsystem. It also describes how each localized module controls its own relationships to the data, such as the booking subsystem performing ORM queries to the database or all of these isolated components streaming encrypted audit logs to the SIEM subsystem at once for complete security coverage.

2.5 Use Cases Diagram

The Use Case Diagram depicts the behavioral structure of the Integrated Hotel Management System, showcasing how different users will interface with the main

components React and FastAPI. The hotel guest is the main human player and has limited access to public-facing functions such as searching inventory, booking reservations, and using the AI chatbot. Operationally, the Receptionist is responsible for daily functions like check-in and check-out processing and guest profile updates in the internal dashboard. At this level, the Hotel Administrator supervises the property and gains staffing skills all while performing top-tier tasks such as room-rate adjustments, revenue report generation, and night audit operations.

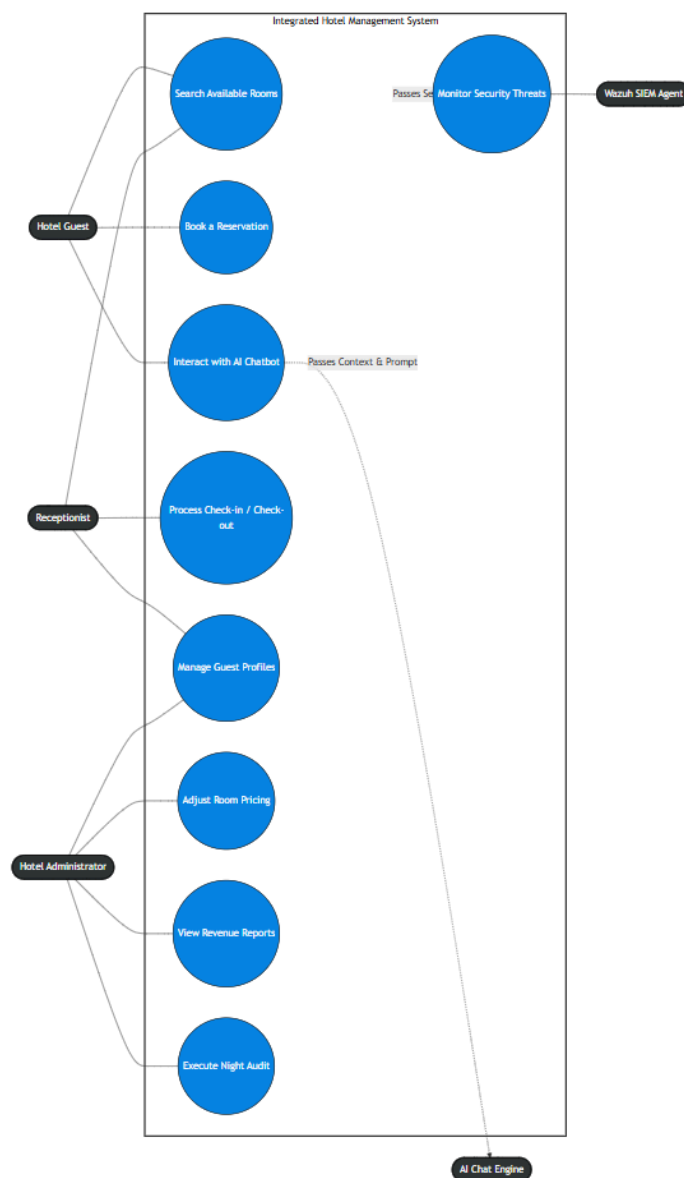


Figure 2.5 Use Case Diagram

Outside of human interactions, the system depends on 2 external automated actors to meet its advanced operational and security needs. For customer support, the AI Chat Engine acts as an intelligence layer; when a guest submits such a query, the backend securely

forwards, including hotel-specific context to that external LLM in order to generate an accurate response. At the same time, the Wazuh SIEM Agent acts as a continuous and active cybersecurity monitor. During the operation of the application, server log and network traffic flows directly to Wazuh agent that will analyze data in real-time detecting logging or blocking malicious activity.

CHAPTER III

STANDARDS USED

3.1 List of the Standard

No	Standard Category	Standards Used	Description / Implementation
1	API Architecture	REST (Representational State Transfer)	Used as the communication standard between the React frontend and the FastAPI backend using HTTP methods (GET, POST, PUT, DELETE).
2	Data Exchange Format	JSON (JavaScript Object Notation)	A standard format used to send and receive data via API request bodies and responses.
3	Authentication & Security	JWT (JSON Web Token) & OAuth2	Implementation of endpoint security using JWT tokens sent via the Authorization: Bearer header.
4	API Documentation	OpenAPI 3.0 (Swagger)	An interactive documentation standard automatically provided by FastAPI at the /docs endpoint.
5	Login Form Format	x-www-form-urlencoded	The standard for transmitting login credentials in accordance with the OAuth 2.0 Password Flow specification.
6	Data Schema Management	Pydantic Models	Used for data type validation and automating the creation of JSON schemas on the backend.
7	Object-Relational Mapping	SQLAlchemy 2.0	A standard ORM used to consistently map Python objects to MySQL database tables.
8	Coding Style	ESLint & PEP 8	Standard formatter to ensure that frontend (JS) and backend (Python) code is clean and easy to read.
9	User Interface Design	Utility-First CSS (Tailwind)	UI design standards that ensure consistency in form factors and responsive layouts across various devices.
10	Security Monitoring (SIEM)Wazuh	Wazuh Standard	Used as a standard for intrusion detection, file integrity monitoring (FIM), and centralized log analysis for a proactive defense architecture.

11	Credential Encryption	Bcrypt (Salted Hashing)	The industry standard for hashing user passwords using the Bcrypt algorithm (bcrypt==4.0.1) before storing them in the database to prevent rainbow table attacks.
12	Rate Limiting Protection	RFC 6585 (SlowAPI)	A standard for limiting the number of requests to the API (particularly login requests) to prevent brute-force and denial-of-service (DoS) attacks.
13	Security Middleware	Secure (HTTP Headers)	Implementation of security standards in HTTP headers such as X-Content-Type-Options, X-Frame-Options, and Content-Security-Policy to prevent XSS and clickjacking attacks.
14	Log Management	JSON Logging / Syslog	A standard log format generated by the application so that it can be efficiently parsed by the Wazuh agent for threat detection.
15	Database Integrity	ACID Properties	MySQL database transaction standards that ensure reservation and payment data remain valid (Atomicity, Consistency, Isolation, Durability) even in the event of a system failure.
16	Modeling Language	UML 2.5 (Unified Modeling Language)	Modeling language standards used to document system designs, including Use Case Diagrams, Class Diagrams, and Sequence Diagrams.

CHAPTER IV

IMPLEMENTATION AND TESTING SCENARIO

4.1 Implementation Description

The system was implemented using a modern architectural approach that separates business logic, the user interface, and cybersecurity systems:

A. Logic and Business Module (Backend):

The system was implemented using the FastAPI framework with a RESTful architecture. To make maintenance easy, each module like `hotel_rooms`, `laundry`, and `night_audit` are separated into their own router. The data is stored in a MySQL database, which is accessed via the SQLAlchemy ORM to ensure transaction consistency (ACID).

B. User Interface and Client Side (Frontend):

Our Frontend is implemented as a responsive single-page application (SPA) with React 19 and styled using Tailwind CSS. We use an interceptor system for our backend communication over Axios, that adds a JWT token to the request header of every call automatically and securely.

C. Proactive Cyber Defense Architecture (Security):

With the deployment of Wazuh SIEM, system security is no longer a passive defense! This implementation includes configuring SlowAPI middleware for rate limiting and SecurityHeadersMiddleware to protect against some of the most common web attacks. Because application logs are sent to Wazuh, they can be analyzed in real-time for the possible brute-force attempts or other unauthorized access.

4.2 Testing Scenarios

This section describes the functional and security testing scenarios organized in a table below specifying explicit steps along with error handling (if any) as requested within the form.

No	Testing Scenario	Detailed Elaboration / Steps	Expected Result (Success)	Alternatives / Error Handling
1	User Login Authentication	The user enters their username and	The system validates the credentials, issues a	If the credentials are incorrect, the system

		password in the login form, then clicks the “Sign In” button.	JWT, and redirects the user to the Dashboard.	displays “Incorrect username or password.” If this occurs repeatedly (>10 times), the system blocks access (Rate Limit).
2	Proactive Security Alert (Wazuh)	The administrator attempts to modify the system configuration file (.env) or access the API without a token.	Wazuh detects file integrity changes (FIM) or unauthorized access and sends a high-priority alert to the security dashboard.	If the connection to Wazuh is lost, the agent will store logs locally (buffering) until the connection is restored.
3	Room Reservation & Pricing	The front desk staff fills out the reservation form by selecting the room type and length of stay.	The system retrieves data from the RoomRateMaster and automatically calculates the total bill.	If the room type is not available on those dates, the system displays a “Room not available” warning.
4	Laundry Service Transaction	Staff add laundry orders through the Laundry module by entering the room number and service type.	The transaction is recorded in the laundry database, and the bill is automatically added to the guest folio for that room.	If the room number is invalid (no guest is staying in that room), the system rejects the transaction.
5	Daily Night Audit Process	The administrator performs the “Night Audit” procedure at the end of the operating day (at 12:00 a.m.).	The system locks today's transactions, posts the daily room charges, and generates a revenue report.	If any transactions are still marked as “Pending,” the system issues a notification to complete those transactions first.
6	Access Control (RBAC)	A user with the ‘Staff’ role attempts	The system detects the user's role and denies	The system returns a “403 Forbidden” response if the

		to access the 'User Management' page, which is intended for Admins.	access or hides the menu from the interface.	user attempts to access the page directly via URL.
7	Database Integrity Protection	The system experienced a sudden loss of internet connection while saving a new reservation.	SQLAlchemy's transaction mechanism automatically rolls back the transaction, ensuring no "garbage" data or partially saved data remains in the database.	The system displays a "Network Error" message and asks the user to try again once the connection is stable.
8	Room Change (Move Room)	The guest requested a room change for technical reasons. The front desk staff processed the room change in the system to a room of the same type.	The status of the old room reverts to 'Dirty/Vacant,' the new room becomes 'Occupied,' and all charges are automatically transferred to the new room.	If the destination room has not yet been cleaned (status 'Dirty'), the system issues a warning to have the room prepared first.
9	Reservation Cancellation & Deposit Refund	A guest cancels a reservation for which a deposit has already been paid. The staff member processes the cancellation in the Hotel Reservations module.	The reservation status changes to 'Cancelled,' and the system records the deposit refund or charges a cancellation fee in accordance with the policy.	The system checks whether the cancellation was made during the non-refundable period; if so, the deposit is automatically non-refundable.
10	Guest Checkout & Folio Settlement	The guest checks out. The front desk clerk verifies the	The system prints the final invoice, changes the room status to	If there are any outstanding charges, the system prevents the checkout process from

		guest bill and processes the final payment.	‘Dirty/Vacant,’ and records the transaction in the revenue report.	completing until the payment status is marked as Settled.
11	Data Validation (SQL Injection Prevention)	Perform security testing by entering invalid characters (such as ' OR 1=1 --) into the guest search field or login form.	The backend (FastAPI) and ORM (SQLAlchemy) validate the input as a plain string, ensuring that no invalid SQL commands are executed.	The system displays an input validation error message without revealing the database structure to the user.
12	Account Receivable (City Ledger)	Transfer corporate guest charges to the Accounts Receivable module for future billing.	Charges are transferred from the Guest Folio to the City Ledger of the relevant company using a unique invoice number.	If the company profile is not registered or the credit limit has been exceeded, the system rejects the transfer of charges.
13	System Performance under Load	Simulating concurrent access by 50 users (Front Office, Laundry, and Housekeeping staff) at the same time.	The FastAPI backend handles requests asynchronously, maintaining a consistent response time of under 2 seconds.	If the server reaches maximum load, the Rate Limiter will prioritize the queue of ongoing transactions.
14	Real-time Housekeeping Update	Housekeeping staff change the room status from “Dirty” to “Clean” via the Housekeeping module.	The room status on the Front Office Dashboard updates instantly without the need to refresh the page.	If a network failure occurs during the update, the system displays a “Sync Failed” notification and prompts staff to try again.

REFERENCES

- [1] Bayer, M. (2024). SQLAlchemy 2.0 Documentation: The Python SQL Toolkit and Object Relational Mapper. SQLAlchemy. <https://docs.sqlalchemy.org/en/20/>
- [2] Langa, S. (2024). FastAPI Documentation: High performance, easy to learn, fast to code, ready for production. FastAPI. <https://fastapi.tiangolo.com/>
- [3] Meta Platforms, Inc. (2024). React: A JavaScript library for building user interfaces. Meta Platforms, Inc. <https://react.dev/>
- [4] Wazuh, Inc. (2024). Wazuh Documentation: Open Source XDR and SIEM for Proactive Cyber Defense. Wazuh. <https://documentation.wazuh.com/>
- [5] SmartBear Software. (2024). OpenAPI Specification 3.0.3. Swagger. <https://swagger.io/specification/>
- [6] Pydantic Services Inc. (2024). Pydantic v2.7 Documentation: Data validation using Python type hints. Pydantic. <https://docs.pydantic.dev/latest/>
- [7] Tailwind Labs Inc. (2024). Tailwind CSS Documentation: Rapidly build modern websites without ever leaving your HTML. Tailwind CSS. <https://tailwindcss.com/docs>
- [8] Python Software Foundation. (2024). PEP 8 – Style Guide for Python Code. Python.org. <https://peps.python.org/pep-0008/>
- [9] OpenJS Foundation. (2024). ESLint Documentation: Find and fix problems in your JavaScript code. ESLint. <https://eslint.org/docs/head/>
- [10] Axios Project. (2024). Axios Documentation: Promise based HTTP client for the browser and node.js. Axios. <https://axios-http.com/docs/intro>